Technische Universität Berlin

Institute of Environmental Technology

Chair of Circular Economy and Recycling Technology

Dynamic MFA workflow with ODYM and the bioDYM project

Lukas Hoppe

2025

1 Introduction bioDYM project

This project offers a use case for dynamic Material Flow Analysis (dMFA) in the field of organic waste management. It is based on the ODYM framework (Pauliuk & Heeren, 2020). The bioDYM project presents studies that apply dMFA with the ODYM framework. Additional features were developed that include:

- Interactive Sankey diagrams
- Interactive stock plots
- Modelling first order processes (e.g. carbon mineralization in soil)
- Monte Carlo Simulation

In this document, the ODYM framework and the general workflow of performing dMFA with ODYM and the bioDYM features is explained. Excerpts of my bachelor thesis that I wrote at the chair in 2022 have been used as foundation for this document.

2 What is ODYM?

In an approach to provide a solution to the problems that researchers face with popular MFA software like STAN (limitations regarding dMFA (Bornhöft, 2017; Cencic & Rechberger, 2008; Pauliuk & Heeren, 2020) and advanced uncertainty evaluation (Laner et al., 2014)), Stefan Pauliuk & Niko Heeren released the *Open software framework for DYnamic Material systems* (ODYM), written in the programming language Python, in 2020 (Pauliuk & Heeren, 2020).

The main motivation for the development of the tool was to establish a modelling framework for data-intensive investigation of complex problems, taking advantage of the intrinsic simplicity of MFA. ODYM only requires the fulfilment of the mass balance principle and the provision of suitable model data, within this framework, many aspects of systems can be explored. With ODYM, different materials, substances, regions, and periods can be simultaneously and dynamically investigated, while enabling advanced uncertainty evaluation of modelling data. (Pauliuk & Heeren, 2020)

According to Pauliuk & Heeren (2020), the unavailability of appropriate, adaptable software led to the situation that MFA is applied by using many different practices and tools. This prevents researchers from working together successfully. Thus, ODYM provides a uniform data format to better facilitate cooperation between research teams. Enabling adaptation and at the same time providing a standardized modelling framework requires establishing a delicate balance between what the authors call “flexibility” and “rigidity”. In order to clearly structure the large number of possible system aspects, the software provides a structure that determines how information has to be inserted into the model database. (Pauliuk & Heeren, 2020)

ODYM is published as Free and Open Source Software (FOSS) (Pauliuk & Heeren, 2020), which allows the reuse of the code within the scope of its license conditions (Manabe et al., 2014; Open Source Initiative, n.d.). While sometimes, the provision of software packages with different FOSS licenses complicates reuse (Manabe et al., 2014), ODYM is published exclusively under the MIT permissive license (Pauliuk & Heeren, 2020). It grants permission to “[…] deal in the Software without

restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so […]" (Open Source Initiative, n.d.). The only condition is to indicate the copyright notice along with the permission notice. In contrast to popular MFA software like STAN, this allows researchers to adapt the software to fit their specific needs. In addition to allowing modification of the framework, ODYM's modular programming structure is intentionally designed to support adaptation (Pauliuk & Heeren, 2020).

Modular software consists of different parts that are at least to some extent independent of each other (Dennis, 1975). They can be combined as needed to address a particular problem, without having to change the code or make use of all software modules (Dennis, 1975). Modularity also enables combining software modules with independently written code to modify and extend the provided program (Dennis, 1975). To serve as a tool to solve different scientific problems in various fields, modularity must be considered while writing code (Kang et al., 2012), which has been done in the case of ODYM (Pauliuk & Heeren, 2020). Thus, modularity enables not just considering only those parameters and model aspects that are relevant but also adapting the framework, like adding a first order model for biological processes that cannot always be described by the use of TCs, as suggested by Urtnowski-Morin et al. (2021). Other features of ODYM are automatic consistency and mass balance checks (Pauliuk & Heeren, 2020).

The software code with explanations along with extensive documentation in the form of a Wiki and tutorials can be accessed via a GitHub repository (Pauliuk & Heeren, 2024). Tutorial No. 5. demonstrates ODYM's suitability for performing extensive dMFA by modelling the global passenger vehicle fleet in 2017, considering the period 1990–2017, 130 countries, relevant processes, and various materials (like steel, Al, plastics, …). For the calculation, integrated dynamic stock modelling functions are used, which allow the determination of the stocks with the help of lifetime functions. The example also includes a Monte Carlo Simulation (MCS) for evaluating the uncertainty regarding the material content and the lifetime of the vehicles. Nonetheless, ODYM does not provide a built-in procedure for MCS, this part of the analysis must be manually programmed depending on the specific use case (Pauliuk & Heeren, 2020). In addition, built-in options for visualization of model results via Sankey diagrams have also not been implemented yet (Pauliuk & Heeren, 2020).

ODYM is an approach to facilitate dMFA of complex systems. It provides a framework for the analysis as well as tools for dynamic stock modelling. The software is free and open, allowing researchers familiar with Python to fit the software to their needs. Although there are other Python MFA tools (e.g. PYMFA 2.1 (Thiébaud, 2019) or DPMFA (Bornhöft, 2017)), ODYM stands out as an approach designed for adaptation as an open science tool.

3 Workflow explained

The Modelling Notebooks of the studies performed present essential characteristics of ODYM and are complemented by specific bioDYM features. The developing process of bioDYM was based on the ODYM tutorials provided in the GitHub repository of the tool (Pauliuk & Heeren, 2024). In fact, the templates could be thought of ODYM tutorials addressing more specific functions helpful for organic waste management but also suggesting general improvements. Due to the framework's

modular design, looking through various tutorial application cases enabled easy adoption of procedures to bioDYM and modifying them. Especially starting steps, like defining the system, setting a modelling period, and choosing relevant system aspects usually include the same initiating routines. Presenting the intended data structure in the tutorials also made transparent where and how extensions can be included. The developers themselves sometimes added calculation steps that have not been implemented yet into ODYM as inherent features (e.g. MCS in Tutorial No. 5.). The detailed documentation including the introduction paper (Pauliuk & Heeren, 2020), a Wiki page, tutorials, and detailed explanations enabled a straightforward working process for this project, making ODYM a prime example for open science.

The workflow is based on the MFA procedure as presented by (Brunner & Rechberger, 2004) (see Figure 1):

Figure 1: MFA procedure flow chart (based on (Laner & Rechberger, 2016; based on (Brunner & Rechberger, 2004))

It should be noted that ODYM is a large modelling framework with many functions. Here, only the features included in the bioDYM project are explained. The modeller is free to use any ODYM or bioDYM features for their dMFA and to customize procedures.

0 Problem and goal definition/Load packages

Problem and goal of the study must be defined before starting the analysis with bioDYM. The Notebook starts with an introduction, and the first coding cell loads the ODYM framework and the bioDYM add-on (both included as local file in the project repository) as well as other required packages.

1 Definition of system and relevant aspects

In this section, the qualitative model is developed. The model period, relevant goods and substances, processes, flows, stocks, and stock changes are defined. The outcome could be likened to the graphical model of STAN. Instead of graphical symbols, ODYM uses lists, tables, dictionaries, and arrays to store information. System aspects (for the bioDYM project time and element) are stored in an index table. Each component (process, flow, stock) is defined by specific attributes: Processes receive an ID and are stored in a list, the IDs are then used to define where flows and stocks occur. Indices, referring to the index table, indicate the dimensions (time and/or element dependency). In the template, goods (e.g. biomass) and substances (e.g. water, carbon, nitrogen) are both called elements. Flows and stocks along with their attributes are stored in separate dictionaries `FlowDict` and `StockDict`. At the end of the section, a value array filled with zeros as placeholder according

to the given dimension is assigned to each flow, stock, and stock change. For the Notebook in the study case_study_bachelor_thesis, the arrays have the shape (21, 2), corresponding to the number of analysis years and elements (total biomass and carbon) (see Figure 2). They are used to store and compute results in the further course of the analysis.

Component values and corresponding calculations are completely built upon arrays, and the considered system aspects define their dimension. For instance, if additionally, three different regions, applying the same treatment process, should be compared in the MFA, the dimension of the arrays would be (21, 3, 2). The index table would then list time, region, and element as system aspects.

```
array([[0., 0.],
       [0., 0.],
       [0., 0.],
       [0., 0.],
       [0., 0.],
       [0., 0.],
       [0., 0.],
       [0., 0.],
       [0., 0.],
       [0., 0.],
       [0., 0.],
       [0., 0.],
       [0., 0.],
       [0., 0.],
       [0., 0.],
       [0., 0.],
       [0., 0.],
       [0., 0.],
       [0., 0.],
       [0., 0.],
       [0., 0.]])
```

Figure 2: Exemplary empty array for the bachelor thesis case study dimensions

The ODYM data structure is responsible for its analytical power. When the system is defined, any calculations can be performed with Python coding. The modelling framework is not limited to a given set of calculation functions. Results must only fit the ODYM data structure at the end of the calculations. The user is free to explore the whole range of Python's potential.

2 Data input into model

Given data can be imported from Excel or other sources. The data import is followed by inserting the data into ODYM's data structure. Input data is assigned to a parameter dictionary `ParameterDict` and empty value arrays get created analogously to the procedure for `StockDict` and `FlowDict`. Also, uncertainty parameters (e.g. standard deviation) can be included.

After defining the system and inserting input data into the model, ODYM enables an automatic consistency check, which checks if dimensions have been used consistently and if flows have been assigned only to existing processes. This feature allows definition errors to be found quickly. The

absence of a graphical user interface makes it more difficult to develop the qualitative model, the consistency check is a way to compensate for this.

At this point, the qualitative MFA system is defined, and an ODYM-compliant memory of all input data has been created. In the next step, analysis results are calculated.

3 MFA Calculations

In this step, the MFA solution is calculated, which can be done in different ways. One very common way of calculating MFA results is by using transfer coefficients (TCs):

$$F\_y\_z = F\_x\_y \cdot TC\_y\_z \tag{1}$$

The arrays of flows and the TCs are combined via the Hadamard product, which allows the multiplication of same dimension matrices according to the scheme:

$$\underset{F\_x\_y}{\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{bmatrix}} \cdot \underset{TC\_y\_z}{\begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \\ b_{31} & b_{32} \end{bmatrix}} = \underset{F\_y\_z}{\begin{bmatrix} a_{11} \cdot b_{11} & a_{12} \cdot b_{12} \\ a_{21} \cdot b_{21} & b_{22} \cdot b_{22} \\ a_{31} \cdot b_{31} & a_{32} \cdot b_{32} \end{bmatrix}} \quad \begin{matrix} \xrightarrow{Elements} \\ \\ \downarrow Years \\ \\ \end{matrix} \tag{2}$$

Other formulas for flow computing can be applied, depending on the input data. If, for instance, TCs on good level are known, but only mass fractions x on substance level are given, the flows F for substances can be calculated via:

$$F_{substance} = F_{good} \cdot x_{substance} \tag{3}$$

Regarding stocks, annual stock changes ($dS(t)$) correspond to the difference of inflow ($F_i(t)$) to outflow ($F_o(t)$), the actual stock is then calculated by summing up all stock changes:

$$dS(t) = F_i(t) - F_o(t) \tag{4}$$

$$S = \sum dS(t) \tag{5}$$

As already explained, ODYM's data structure allows including any Python operation within the MFA. Therefore, more complex modelling steps can be easily integrated. The idea is to make use of the modular structure of the analysis. If the qualitative model is set up, the modeller can use different ways of calculating for different parts of the system. For this project, dynamic stock modelling (ODYM inherent) and first order processes have been included. If a production process leads to the flow of products with different lifetimes to the use phase, the corresponding stock can be computed with survival functions. Dynamic stock modelling is performed in the rye straw cascading system in the bioDYM project. Detailed explanations can be found in the ODYM tutorial provided in the

framework's documentation (ODYM-master/docs/ODYM Example and Tutorial No. 3. Dynamic Stock modelling.ipynb)

The calculations of first order modelling processes (FOMPs) are separated from the standard ODYM procedure. In the bioDYM project, a first order model is used to calculate the mineralization of organic carbon in soils. This procedure is included both, in the bachelor thesis study and the rye straw cascading study. The FOMP-associated parameters are stored in a separate dictionary `BioParameterDict`. After calculating the part of the system independent of FOMPs, the standard ODYM procedure is put on pause, a new model is defined with separate practices to calculate FOMPs and the results are fitted to ODYM's data structure and subsequently, the ODYM procedure is continued. This technique can be applied to any number of FOMPs in a system. The first order model is also built upon array calculations. Extensive looping through each year and element to apply the exponential function is more intuitive, but very inefficient. With regard to the MCS (where calculations are run 10,000 times or more), a fast approach had to be chosen. The first order exponential function is used to calculate the relative course of the decay process through the years (d_{year}), which can then be used for all cohort inflows F_{ic}. The computation follows:

$$\begin{matrix} & \xrightarrow{Cohort\text{-}Years} \\ Years \downarrow & \end{matrix} \begin{bmatrix} F_{i1} & 0 & 0 \\ F_{i1} & F_{i2} & 0 \\ F_{i1} & F_{i2} & F_{i3} \end{bmatrix} \wedge \begin{bmatrix} d_1 \\ d_2 \\ d_3 \end{bmatrix} = \begin{bmatrix} F_{i1} \cdot d_1 & 0 & 0 \\ F_{i1} \cdot d_2 & F_{i2} \cdot d_1 & 0 \\ F_{i1} \cdot d_3 & F_{i2} \cdot d_2 & F_{i3} \cdot d_1 \end{bmatrix} \quad (6)$$

F_{i1} is the inflow into the FOMP in year 1 and the first column of the result array represents the first order decay of this inflow cohort. F_{i2} is the inflow in year 2, which is not yet present in year 1, thus, the decay process starts in year 2 with d_1. In the case study, summing up each row of the result array leads to the total stock of the process in each year, since the applied exponential equation represents the share of remaining carbon during the mineralization process. However, this calculation approach is not limited to stock calculations. For instance, a first order model that represents emissions could also be used (e.g. first order model for methane emissions from landfills (Towprayoon et al., 2019)). Summing up array rows would then represent the annual emission outflow of the process.

In the case study, after computing the soil stock and stock change for each year, Equation (4) (p. 5) is rearranged to obtain the outflows:

$$F_o(t) = F_i(t) - dS(t) \quad (7)$$

Results are then inserted into ODYM's `StockDict` and `FlowDict`.

ODYM provides an automatic mass balance check that reports the balance dimension and all absolute balancing errors per process. Note that, due to computer hardware arithmetic, even balanced processes can have very small balancing errors shown (e.g. 10^{-12}) (Python Software Foundation, 2008).

Using uncertain input data inevitably leads to uncertain results. This can be quantified by “uncertainty propagation” (Zhang, 2021). In MFA, Monte Carlo Simulation (MCS) is frequently used to assess the effects of data uncertainty on the model results (Brunner & Rechberger, 2016). MCS includes random sampling of input variables according to their probability density function and running the simulation with the obtained values to see how they influence the outputs (Zhang, 2021). Computer-based random number generation is well tested and considered to be pseudo-random, since the underlying algorithms are usually reproducible (Harrison, 2010). For each run in the MCS, a new set of pseudo-random values for all variables with uncertainty is provided and results are collected afterward (Harrison, 2010). Approximated probability density functions of output results can be generated by analysing the simulation outcome (Raychaudhuri, 2008). It is important to note that drawing random values may lead to breaking the mass balance principle for Monte Carlo model runs. If you, e.g., sample transfer coefficients that have to sum up to one to keep the mass balance correct, random values will almost certainly introduce small errors. This is why data reconciliation is an essential step to consider while doing MCS. Data reconciliation is a way to process your Monte Carlo data set that the mass balance principle remains fulfilled (Brunner & Rechberger, 2016). To determine the number of simulation runs that should be done, it is a common procedure to choose a number of runs, and then check if the mean and standard error of the results change significantly with additional model runs, or if the scale of deviation is acceptable (Lerche & Mudford, 2005). (Brunner & Rechberger, 2016)

MCS is included in the bachelor thesis study: The simulation repeats all the preceding calculations but introduces another dimension: according to the given standard deviation and, assuming a log-normal distribution for each parameter, 10,000 pseudo-random values are drawn. The dimension of the arrays is now (10,000, 21, 2) and they are stored in separate MCS dictionaries. The calculations done to compute the initial results are then repeated but with the MCS dictionaries and flow and stock results (now also in the dimension (10,000, 21, 2)) are again separately stored. More specific distributions functions like Weibull or gamma distribution can be integrated into the template. The simulation closes by calculating the absolute standard deviation for all stock and flow values for the Excel export.

4 Presentation (+ Interpretation) of results

All flows and stocks can be plotted via simple plots. ODYM originally does not include procedures for interactive Sankey diagrams or stock plots (Pauliuk & Heeren, 2020). The bioDYM modelling template includes an approach to provide options for dynamic and interactive Sankey diagrams and stock plots. The method makes use of ODYM’s dictionaries and extracts all information needed to build the diagrams. The developed approach is inspired by an application of the European Union’s statistical office that visualises material flows within the EU as interactive Sankey diagrams (European Commission, 2020).

MCS results can be displayed as violin plots and simple plots of all model runs.

In a last step, MFA results of all flows and stocks are exported to Excel. Additionally, mean values and absolute standard deviation of the MCS may also be exported as a separate file.

References

Bornhöft, N. A. (2017). *A dynamic probabilistic material flow modeling method for environmental exposure assessment*. https://doi.org/10.5167/UZH-152424

Brunner, P. H., & Rechberger, H. (2004). Practical handbook of material flow analysis. *The International Journal of Life Cycle Assessment*, *9*(5), 337–338. https://doi.org/10.1007/BF02979426

Brunner, P. H., & Rechberger, H. (2016). *Handbook of Material Flow Analysis* (2nd ed.). CRC Press. https://doi.org/10.1201/9781315313450

Cencic, O., & Rechberger, H. (2008). Material flow analysis with Software STAN. *Journal of Environmental Engineering and Management*, *18*, 3–7.

Dennis, J. B. (1975). Modularity. *Software Engineering*, 128–182.

European Commission. (2020). *Circular economy flow diagrams*. https://ec.europa.eu/eurostat/cache/sankey/circular_economy/sankey.html

Harrison, R. L. (2010). Introduction To Monte Carlo Simulation. *AIP Conference Proceedings*, *1204*, 17–21. PubMed. https://doi.org/10.1063/1.3295638

Kang, P., Selvarasu, N., Ramakrishnan, N., Ribbens, C., Tafti, D., Cao, Y., & Varadarajan, S. (2012). Implementing modular adaptation of scientific software. *Journal of Computational Science*, *3*, 28–45. https://doi.org/10.1016/j.jocs.2012.01.007

Laner, D., Rechberger, H., & Astrup, T. (2014). Systematic Evaluation of Uncertainty in Material Flow Analysis: Uncertainty Analysis in Material Flow Analysis. *Journal of Industrial Ecology*, *18*(6), 859–870. https://doi.org/10.1111/jiec.12143

Lerche, I., & Mudford, B. S. (2005). How Many Monte Carlo Simulations Does One Need to Do? *Energy Exploration & Exploitation*, *23*(6), 405–427. https://doi.org/10.1260/014459805776986876

Manabe, Y., German, D. M., & Inoue, K. (2014). Analyzing the Relationship between the License of Packages and Their Files in Free and Open Source Software. In L. Corral, A. Sillitti, G. Succi, J. Vlasenko, & A. I. Wasserman (Eds.), *Open Source Software: Mobile Open Source Technologies* (pp. 51–60). Springer Berlin Heidelberg.

Open Source Initiative. (n.d.). *The MIT License*. Retrieved 9 May 2022, from https://opensource.org/licenses/MIT

Pauliuk, S., & Heeren, N. (2020). ODYM—An open software framework for studying dynamic material systems: Principles, implementation, and data structures. *Journal of Industrial Ecology*, *24*(3), 446–458. https://doi.org/10.1111/jiec.12952

Pauliuk, S., & Heeren, N. (2024). *ODYM* [Computer software]. https://github.com/IndEcol/ODYM

Python Software Foundation. (2008). *3.10.5 Documentation » The Python Tutorial » 15. Floating Point Arithmetic: Issues and Limitations*. https://docs.python.org/3/tutorial/floatingpoint.html

Raychaudhuri, S. (2008). Introduction to Monte Carlo simulation. *2008 Winter Simulation Conference*, 91–100. https://doi.org/10.1109/WSC.2008.4736059

Thiébaud, E. (2019). *PYMFA 2.1—A A probabilistic dynamic material flow analysis tool—User manual*. Technology and Society Lab, Empa. https://bitbucket.org/Xeelk/pymfa2/raw/333fc431b563ce425f1a950f17022e0d60c51ec6/Manual%20-%20ReadMe/Pymfa2.1_Manual.pdf

Towprayoon, S., Ishigaki, T., Chiemchaisri, C., & Abdel-Aziz, A. O. (2019). *2019 Refinement to the 2006 IPCC Guidelines for National Greenhouse Gas Inventories*. IPCC. https://www.ipcc-nggip.iges.or.jp/public/2019rf/pdf/5_Volume5/19R_V5_3_Ch03_SWDS.pdf

Urtnowski-Morin, C., Tanguay-Rioux, F., Legros, R., & Spreutels, L. (2021). Upgrading waste material flow analysis with process models: The case of anaerobic digestion. *Journal of Cleaner Production*, *298*, 126695. https://doi.org/10.1016/j.jclepro.2021.126695

Zhang, J. (2021). Modern Monte Carlo methods for efficient uncertainty quantification and propagation: A survey. *WIREs Computational Statistics*, *13*(5), e1539. https://doi.org/10.1002/wics.1539